Processingをはじめよう

第8章

関数

目次

- 関数の基礎
- 関数を作る
- 値を返す

関数の基礎

- Example 8-1 (EX_09_01)
  - １から指定した数までの乱数を返す関数 rollDice()を作る・使う
  - int型へ変換 int() P201
    - 1からnumSidesの乱数（整数）
      1 + int(random(numSides))
- Example 8-2 (EX_09_02)
  - Example8-1を関数を使わずに書いている

Example 8-1・8-2

```
// Example 8-1 (EX_09_01)
void setup() {
  println("Ready to roll!");
  rollDice(20);
  rollDice(20);
  rollDice(6);
  println("Finished.");
}

void rollDice(int numSides) {
  int d = 1 + int(random(numSides));
  println("Rolling... " + d);
}
```

関数 rollDice()を使う（呼びだす）

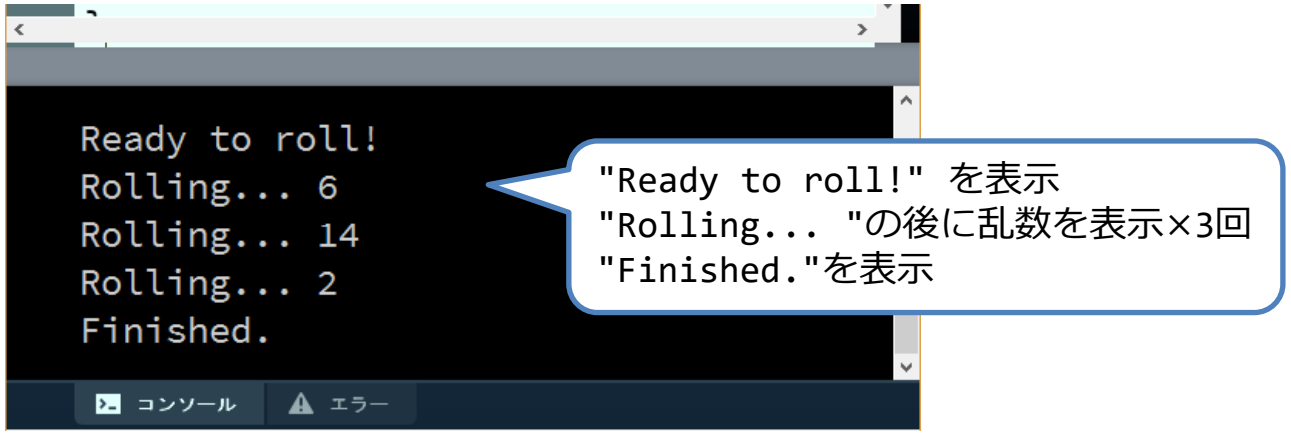
関数 rollDice()を作る（定義）
この関数は「サイコロを振る」という名前

```
// Example 8-2 (EX_09_02)
void setup() {
  println("Ready to roll!");
  int d1 = 1 + int(random(20));
  println("Rolling... " + d1);
  int d2 = 1 + int(random(20));
  println("Rolling... " + d2);
  int d3 = 1 + int(random(6));
  println("Rolling... " + d3);
  println("Finished.");
}
```

共通する処理を3回
繰り返し実行している

共通する処理が複数回繰り返している
↓
共通部分を関数を使って
再利用できる形にまとめる

実行結果



- 複数の命令をまとめて関数を作る
  - 処理内容が分かりやすく理解できる名前をつける
  - 同じような処理を何度も書かずに済む
  - 1度作った関数は何度も使える

関数の定義

- 関数の定義

複数の引数を指定するときは
引数の型 引数名を
コンマで区切って書く

```
戻り値の型　関数名(引数の型 引数名, ...){
    文;
    ...
    return 式;
}
```

return 式; は
戻り値がない場合(void型)は省略

- 例 Example 8-1

```
void rollDice(int numSides){
  int d = 1 + int(random(sumSides));
  println("Rolling... " + d);
}
```

戻り値の型	void
関数名	rollDice
引数の型	int
引数名	numSides

引数numSidesを使って
処理を行う

関数の定義

- 例 Example 8-5

引数リスト　int x, int y

```
void owl(int x, int y){
  pushMatrix();
   (省略)
  popMatrix();
}
```

- 例 Example 8-8

```
float calculateMars(float w) {
  float newWeight = w * 0.38;
  return newWeight;
}
```

戻り値がある場合は
return 式; で値を返す

関数の定義

- 名前の付け方について
  - 英字・数字・アンダースコアなどを使える
    - ただし数字から始められない
  - 長さ制限なし
  - 予約語は使えない
  - 大文字・小文字は区別する

関数を呼び出す

- 関数の呼び出し方

```
関数名(引数);
```

- 例 Example 8-1

```
void setup(){
  println("Ready to roll!");
  rollDice(20);
  rollDice(20);
  rollDice(6);
}
```

引数で値を渡して
関数を呼び出す

関数を作る

- Example 8-3 (EX_09_03)
  - フクロウを描く（関数を使用しない例）
    - 複数の図形や色・線の設定などでフクロウを表現
- Example 8-4 (EX_09_04)
  - フクロウを2羽描く（関数を使用しない例）
    - フクロウを2羽描くために共通した処理を2回実行
- Example 8-5 (EX_09_05)
  - フクロウを描く関数 owl()
    - 関数owl()を作り、再利用して2羽描く

Example 8-3

フクロウを描く処理

```
void setup() {
  size(480, 120);
}

void draw() {
  background(176, 204, 226);          // 背景色の設定
  translate(110, 110);                // 座標を(110, 110)移動
  stroke(138, 138, 125);              // 線の色を設定
  strokeWeight(70);                   // 線の太さを70に設定
  line(0, -35, 0, -65);               // 体の部分（線を描く）
  noStroke();                         // 図形の縁取りを消す
  fill(255);                          // 図形の色を白に設定
  ellipse(-17.5, -65, 35, 35);        // 円を描く（左目の周り）
  ellipse(17.5, -65, 35, 35);         // 円を描く（右目の周り）
  arc(0, -65, 70, 70, 0, PI);         // 円弧を描く（下あご）
  fill(51, 51, 30);                   // 図形の色を設定
  ellipse(-14, -65, 8, 8);            // 円を描く（左目）
  ellipse(14, -65, 8, 8);             // 円を描く（右目）
  quad(0, -58, 4, -51, 0, -44, -4, -51);  // 四辺形を描く（くちばし）
}
```

Example 8-4

```
void setup() {
  size(480, 120);
}
void draw() {
  background(176, 204, 226);

  // 左のフクロウ
  translate(110, 110);
  stroke(138, 138, 125);
  strokeWeight(70);
  line(0, -35, 0, -65);
  noStroke();
  fill(255);
  ellipse(-17.5, -65, 35, 35);
  ellipse(17.5, -65, 35, 35);
  arc(0, -65, 70, 70, 0, PI);
  fill(51, 51, 30);
  ellipse(-14, -65, 8, 8);
  ellipse(14, -65, 8, 8);
  quad(0, -58, 4, -51, 0, -44, -4, -51);
```

フクロウを描く処理

```
// 右のフクロウ
 translate(70, 0);
 stroke(138, 138, 125);
 strokeWeight(70);
 line(0, -35, 0, -65);
 noStroke();
 fill(255);
 ellipse(-17.5, -65, 35, 35);
 ellipse(17.5, -65, 35, 35);
 arc(0, -65, 70, 70, 0, PI);
 fill(51, 51, 30);
 ellipse(-14, -65, 8, 8);
 ellipse(14, -65, 8, 8);
 quad(0, -58, 4, -51, 0, -44, -4, -51);

}
```

フクロウを描く処理

フクロウを描く処理が２回繰り返している
↓
共通部分（フクロウを描く）を関数を使って
再利用できる形にまとめる

Example 8-5

```
void setup() {
  size(480, 120);
}

void draw() {
  background(176, 204, 226);
  owl(110, 110);
  owl(180, 110);
}
```

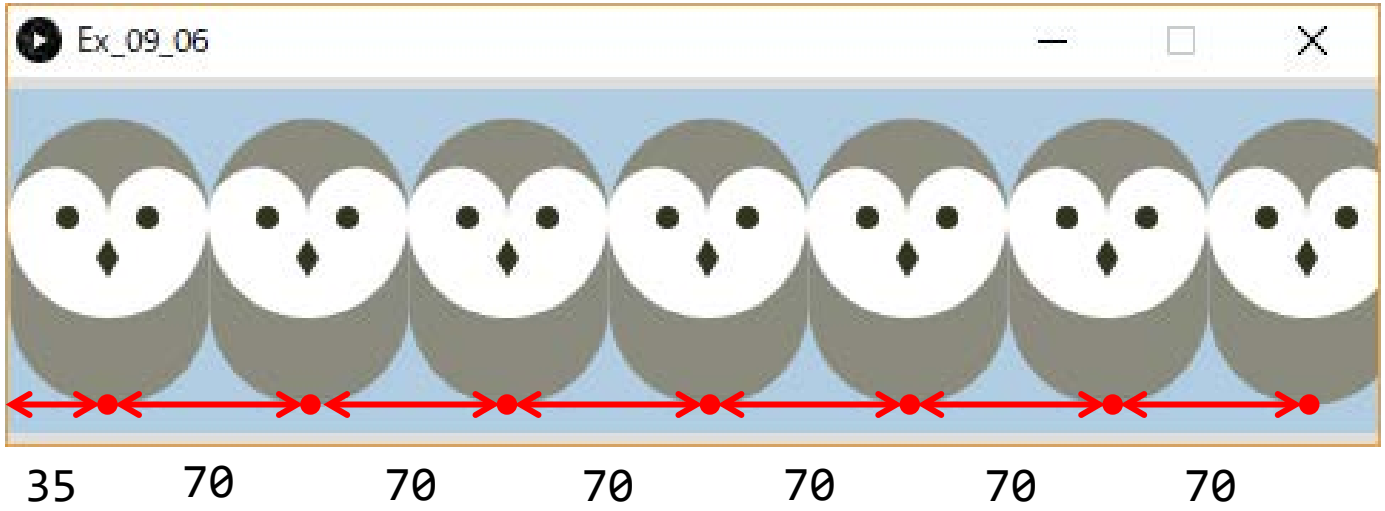
関数を使う
- (110, 110)にフクロウを描く
- (180, 110)にフクロウを描く

```
void owl(int x, int y) {
 pushMatrix();        // 座標を保存（追加）
 translate(x, y);
 stroke(138, 138, 125);
 strokeWeight(70);
 line(0, -35, 0, -65);
 noStroke();
 fill(255);
 ellipse(-17.5, -65, 35, 35);
 ellipse(17.5, -65, 35, 35);
 arc(0, -65, 70, 70, 0, PI);
 fill(51, 51, 30);
 ellipse(-14, -65, 8, 8);
 ellipse(14, -65, 8, 8);
 quad(0, -58, 4, -51, 0, -44, -4, -51);
 popMatrix();         // 座標を復元（追加）
}
```

関数 owl(int x, int y)
フクロウの位置(x,y)を指定して
そこにフクロウを描く関数

関数を作る

- Example 8-6 (EX_09_06)
  - 関数を使って複数のフクロウを描く
  - for文使って繰り返し関数を呼び出す



Example 8-6

```
void setup() {
  size(480, 120);
}

void draw() {
  background(176, 204, 226);
  for (int x = 35; x < width + 70; x += 70) {
    owl(x, 110);
  }
}
```

for文を使って
繰り返し関数を呼び出す

```
void owl(int x, int y) {
  pushMatrix();
  translate(x, y);
  stroke(138, 138, 125);
  strokeWeight(70);
  line(0, -35, 0, -65);
  noStroke();
  fill(255);
  ellipse(-17.5, -65, 35, 35);
  ellipse(17.5, -65, 35, 35);
  arc(0, -65, 70, 70, 0, PI);
  fill(51, 51, 30);
  ellipse(-14, -65, 8, 8);
  ellipse(14, -65, 8, 8);
  quad(0, -58, 4, -51, 0, -44, -4, -51);
  popMatrix();
}
```

関数 owl(int x, int y)
Example 8-5と同じ

関数を作る

- Example 8-7 (EX_09_07)
  - 関数を使い大きさや色の異なるフクロウを描く
  - 関数の一部を修正
    - 色を指定できるパラメータ追加　　`int g`
    - 大きさを指定できるパラメータ追加　　`float s`

Example 8-7

```
void setup() {
  size(480, 120);
}

void draw() {
  background(176, 204, 226);
  randomSeed(0);
  for (int i = 35; i < width + 40; i += 40) {
    int gray = int(random(0, 102));
    float scalar = random(0.25, 1.0);
    owl(i, 110, gray, scalar);
  }
}
```

座標、色、大きさを指定してフクロウを描く

関数 owl(int x, int y, int g, float g)
色と大きさを指定できるように修正

```
void owl(int x, int y, int g, float s) {
  pushMatrix();
  translate(x, y);
  scale(s);                          // s倍に伸縮
  stroke(138-g, 138-g, 125-g);       // 色を指定
  strokeWeight(70);
  line(0, -35, 0, -65);
  noStroke();
  fill(255);
  ellipse(-17.5, -65, 35, 35);
  ellipse(17.5, -65, 35, 35);
  arc(0, -65, 70, 70, 0, PI);
  fill(51, 51, 30);
  ellipse(-14, -65, 8, 8);
  ellipse(14, -65, 8, 8);
  quad(0, -58, 4, -51, 0, -44, -4, -51);
  popMatrix();
}
```

色の指定方法が
教科書と一部異なる

値を返す

- Example 8-8 (EX_09_08)
  - 火星表面における体重を計算
    体重×0.38倍
  - 体重をパラメータとして入力、計算結果を返す
    関数calcuateMars()

Example 8-8

```
void setup() {
  float yourWeight = 132;
  float marsWeight = calculateMars(yourWeight);
  println(marsWeight);
}

float calculateMars(float w) {
  float newWeight = w * 0.38;
  return newWeight;
}
```